VECTOR

MICROSAR vMem

Technical Reference

vMem_30_Tc3xxInf01

Version 2.00.00

Authors	Charlotte Ricke, Tobias Schwindl, Bethina Mausz
Status	Released

Document Information

History

Author	Date	Version	Remarks
Charlotte Ricke	2018-06-05	1.00.00	Initial creation
Charlotte Ricke	2018-07-17	1.01.00	STORYC-5975: support alternate address map of Tc3xx in vMem
Tobias Schwindl Charlotte Ricke	2018-10-05 2019-02-21	1.02.00	STORYC-4938: Support write and erase burst in vMem_30_Tc3xxInf01 STORY-8867: vMem for Tc3xx shall support Os_Peripheral API
Bethina Mausz Tobias Schwindl	2019-03-20	1.03.00	STORY-10827 - vMem for Tc3xx shall provide service to check for erased flash as hardware specific function
Tobias Schwindl	2019-06-11	1.03.01	STORY-12869: vMem Tc3xx shall support PFlash accesses from the HSM Core
Charlotte Land	2019-07-03	1.04.00	STORY-9222: Implement Replace Logical Sector feature in vMem for Tc3xx
Tobias Schwindl	2019-12-10	1.05.00	MWDG-2501: vMem_Tc3xx shall provide a service to check for hardware error states for latest job
Tobias Schwindl Bethina Mausz	2020-01-02	2.00.00	MWDG-1902: RAM Alignment check within vMem_Tc3xxInf01 MWDG-6: vMem for Tc3xx shall provide service to check for erased flash

Reference Documents

No.	Source	Title	Version
[1]	AUTOSAR	AUTOSAR_SWS_DevelopmentErrorTracer.pdf	3.2.0
[2]	Vector	TechnicalReference_vMem.pdf	see delivery
[3]	Infineon	AURIX TC3xx	2.5.1
[4]	Vector	TechnicalReference_Os.pdf	see delivery
[5]	Infineon	HSM_TS_V2.0_Rev2.0.pdf	2.0
[6]	Vector	TechnicalReference_vMemAccM.pdf	see delivery

Scope of the Document

This technical reference describes the specific use of the vMem_30_Tc3xxInf01 driver software. It supplements the general vMem driver technical reference [2].

The vMem_30_Tc3xxInf01 driver was tested with AURIX TC397 B-Step, which is considered as a reference for the entire AURIX TC3xx family.



Caution
We have configured the programs in accordance with your specifications in the questionnaire. Whereas the programs do support other configurations than the one specified in your questionnaire, Vector´s release of the programs delivered to your company is expressly restricted to the configuration you have specified in the questionnaire.

© 2020 Vector Informatik GmbH

Contents

1 Component History 7

2 Introduction 8
2.1 Architecture Overview 8
2.2 Component Interfaces 9

3 Functional Description 10
3.1 Features 10
3.1.1 Additions/ Extensions 10
3.1.2 Limitations 10
3.1.2.1 Memory Instances 10
3.1.2.2 ECC Error Detection 10
3.1.2.3 RAM alignment 10
3.2 Initialization 10
3.3 Main Functions 10
3.4 Support of Software Download and Address Mapping 11
3.5 Replace Logical Sector 12
3.6 Error Handling 13
3.6.1 Development Error Reporting 13
3.7 Hardware Software Interface 13
3.7.1 Data Memory Unit (DMU) 13
3.7.2 System Memory Unit (SCU) 13
3.7.3 User Configuration Block (UCB) 14
3.7.4 Compiler Abstraction 14
3.7.5 HSM Bridge 14
3.8 Safe Context: OS peripheral access API 14

4 Integration 16
4.1 Scope of Delivery 16
4.2 Critical Sections 16
4.3 HSM Integration 17

5 API Description 18
5.1 Type Definitions 18
5.2 Services provided by vMem_30_Tc3xxInf01 20
5.2.1 vMem_30_Tc3xxInf01_GetActiveAddressRegion 20
5.2.2 vMem_30_Tc3xxInf01_ReplaceLogicalSector 20
5.2.3 vMem_30_Tc3xxInf01_GetLastErrorState 21
5.3 Services used by vMem_30_Tc3xxInf01 22

5.4 Callout Functions 22
5.4.1 vMem_30_Tc3xxInf01_Unlock 22
5.4.2 vMem_30_Tc3xxInf01_Lock 23

6 Configuration 24
6.1 Configuration of Alternate Address Mapping 24
6.2 Configuration of Write and Erase Burst Feature 25
6.2.1 Limitations for Erase Burst 25
6.2.2 Limitations for Write Burst 25

7 Glossary and Abbreviations 26
7.1 Glossary 26
7.2 Abbreviations 26

8 Contact 27

Illustrations

Figure 2-1 Interfaces to adjacent modules of vMem_30_Tc3xxInf01 9
Figure 6-1 Example configuration for TC39x 24

Tables

Table 1-1 Component history 7
Table 3-1 Features provided beyond the vMem standard 10
Table 3-2 Service IDs 13
Table 3-4 Peripheral Region HCSI 14
Table 3-5 Peripheral Region DMU 15
Table 3-6 Peripheral Region SCU 15
Table 3-7 Peripheral Region UCB 15
Table 4-1 Additional static files 16
Table 4-2 Additional Generated files 16
Table 5-1 Type definitions of extended hardware specific functionality 19
Table 5-2 vMem_30_Tc3xxInf01_ReplaceSectorType 19
Table 5-3 vMem_30_Tc3xxInf01_GetActiveAddressRegion 20
Table 5-4 vMem_30_Tc3xxInf01_ReplaceLogicalSector 21
Table 5-5 vMem_30_Tc3xxInf01_GetLastErrorState 22
Table 5-6 Services used by the vMem_30_Tc3xxInf01 22
Table 5-7 vMem_30_Tc3xxInf01_Unlock 23
Table 5-8 vMem_30_Tc3xxInf01_Lock 23
Table 7-1 Glossary 26
Table 7-2 Abbreviations 26

1 Component History

The component history gives an overview over the important milestones that are supported in the different versions of the component.

Component Version	New Features
1.00.00	Creation of component
1.01.00	Added extended functionality: get active address region.
1.02.00	Support alternate address map of Tc3xx in vMem.
1.04.00	Support write and erase burst in vMem_30_Tc3xxInf01. Support usage of OS peripheral access API.
1.05.00	Support service to check for erased flash (blank check).
1.06.00	Support service to replace a logical sector.
1.07.00	Support service to get detailed error information for last request.
2.00.00	Support of: RAM alignment check, IsBlank service and removed hardware-specific blankCheck service.

Table 1-1 Component history

2 Introduction

This document describes the functionality, API and configuration of the MICROSAR BSW module vMem_30_Tc3xxInf01.

Supported AUTOSAR Release:	4	
Supported Configuration Variants:	pre-compile	
Vendor ID:	VMEM_30_TC3XXINF01_VENDOR_ID	30 decimal (= Vector-Informatik, according to HIS)
Module ID:	VMEM_30_TC3XXINF01_MODULE_ID	255 decimal

The vMem_30_Tc3xxInf01 is the driver for the Infineon AURIX TC3xx family program flash. The driver can also be used to program the HSM program flash. See document [3] for a detailed description of the functionality of entire AURIX TC3xx Platform.

This technical reference describes the specific use of the vMem_30_Tc3xxInf01 software. It supplements the general vMem driver technical reference [2]. Note that the names of all services, files, types and defines that are described in [2] are infixed in the implementation of the vMem_30_Tc3xxInf01 with `vMem_30_Tc3xxInf01`.

2.1 Architecture Overview

Refer to [2].

2.2 Component Interfaces

The next figure shows the interfaces to adjacent modules of the vMem_30_Tc3xxInf01. Note that the infix of all interfaces of this specific driver is `vMem_30_Tc3xxInf01`.

The interfaces of the extended functionality are described in chapter 5. All standard interfaces (e.g. Read, Write, Erase, IsBlank and GetJobResult) are described in [2].

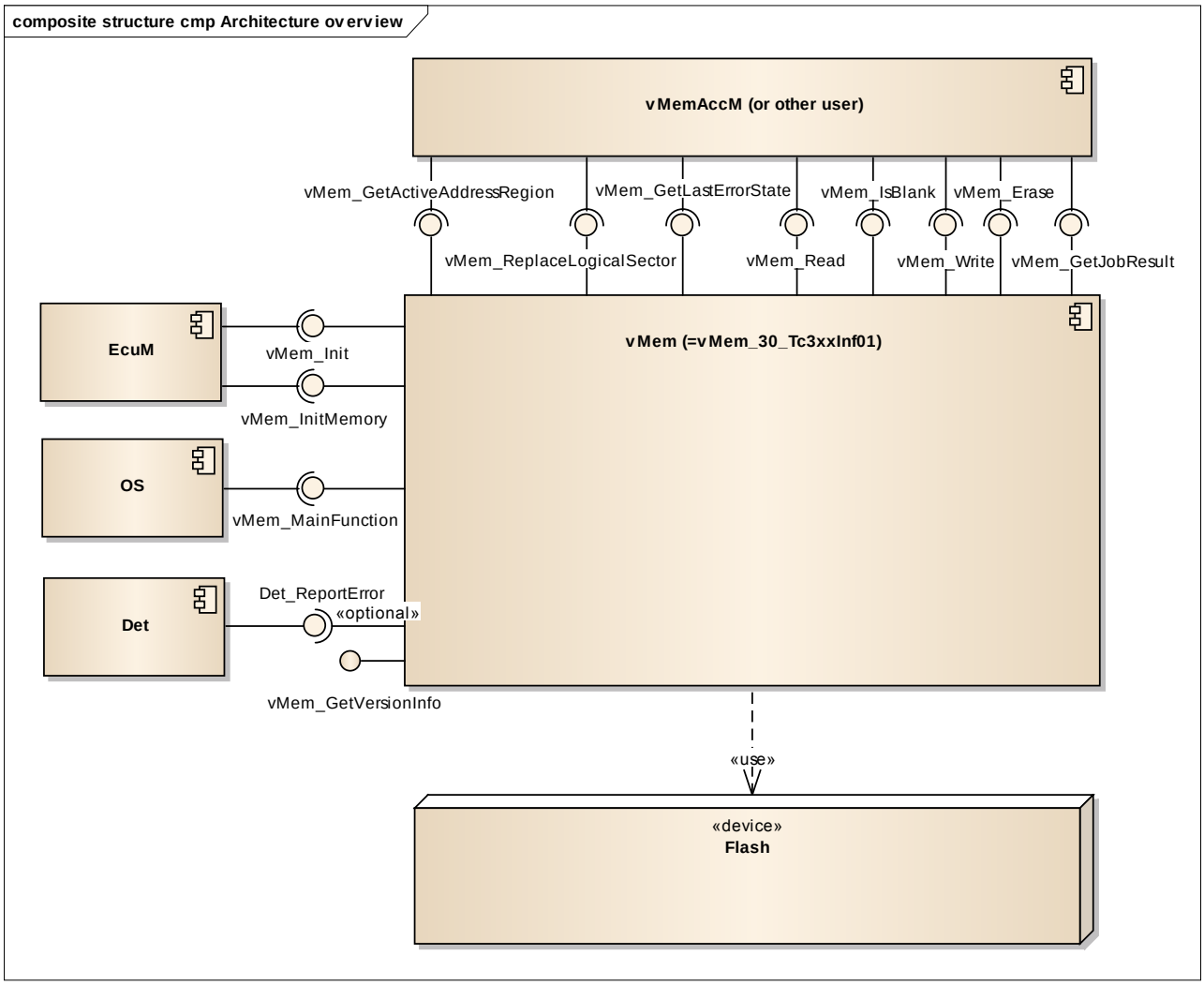


Figure 2-1 Interfaces to adjacent modules of vMem_30_Tc3xxInf01

3 Functional Description

3.1 Features

The features listed in the following table cover only the extended hardware specific functionality for the vMem_30_Tc3xxInf01. For the standard functionality, which is provided by all vMem drivers, see [2].

3.1.1 Additions/ Extensions

The following features are provided beyond the vMem standard features:

Features Provided Beyond The vMem Standard
Get the currently active address region.
Replace a logical sector.
Get detailed error information of last vMem request.

Table 3-1 Features provided beyond the vMem standard

3.1.2 Limitations

3.1.2.1 Memory Instances

The vMem_30_Tc3xxInf01 manages only one memory instance. Therefore, the instance identifier, which is passed as a parameter in all services, is not used by the vMem_30_Tc3xxInf01, except for parameter checks.

3.1.2.2 ECC Error Detection

The vMem_30_Tc3xxInf01 does not evaluate the ECC error status and in particular does not return the results `VMEM_READ_CORRECTED_ERRORS` and `VMEM_READ_UNCORRECTABLE_ERRORS`.

In order to inform higher layers of ECC errors, an ECC interrupt handler can use a notification provided by the vMemAccM as described in [6].

3.1.2.3 RAM alignment

The address of the buffer used for write operations has to be aligned to two or (better) to four Bytes (leading to atomic bus transactions). This has to be ensured by the environment, because the vMem_30_Tc3xxInf01 does not take care of the RAM alignment by itself, but will only report a DET error to the user. For information on the RAM alignment see [3] chapter 5.3.4.10 (Atomicity of Data Accesses).

3.2 Initialization

Refer to [2].

3.3 Main Functions

The vMem_30_Tc3xxInf01 has one central processing function `vMem_30_Tc3xxInf01_MainFunction()`, which can be called with a fixed cycle time.

When an asynchronous request is accepted, its processing will be handled within the `vMem_30_Tc3xxInf01_MainFunction()`. The vMem_30_Tc3xxInf01 accepts at most

one asynchronous job request at a time. The following are the asynchronous API services of the vMem_30_Tc3xxInf01:

- `vMem_30_Tc3xxInf01_Write()`
- `vMem_30_Tc3xxInf01_Erase()`
- `vMem_30_Tc3xxInf01_ReplaceLogicalSector()`



Note
The `vMem_30_Tc3xxInf01_Read` job is performed synchronously as memory mapped read. Also, the `vMem_30_Tc3xxInf01_IsBlank()` job is performed synchronously.

3.4 Support of Software Download and Address Mapping

In order to support Software Download (OTA) the TC3xx devices (except TC33x and TC33xED) provide the possibility to split the entire program flash into two address regions, A and B (see [3], note that in [3] the address regions are referred to as ‘banks’). While the program is executed from the active address region, new software can be written to the inactive address region. When the update is completed, the address regions can be “swapped” so that the new software is executed after the next system reset. This implies that the address mapping will change, so that no data needs to be copied and the address ranges being executed from are always the same.

Therefore, if Software Download is enabled, the TC3xx platform uses two different address mappings: the standard address map and the alternate address map. The standard address map describes the actual physical addresses of the program flash. In the standard address map, address region A is the active address region. The alternate address map describes the program flash for read/code execution, when a swap has been completed from the standard address map. In the alternate address map, address region B is the active address region.

Note, that for write and erase operations the address mapping is always described in the standard address map. The user of the vMem_30_Tc3xxInf01 shall be able to use the same address map for read, write and erase operations. Therefore, the vMem_30_Tc3xxInf01 performs a remapping of the read addresses in case the alternate address map is active. To enable the vMem_30_Tc3xxInf01 to do so, the alternate address mapping must be configured in the DaVinci Configurator 5. See chapter 6.1 for more details on the configuration.



Basic Knowledge
The user of the vMem_30_Tc3xxInf01 can always use the standard address mapping, i.e. the physical addresses of the program flash, regardless of swap settings. For this purpose, the alternate address mapping must be configured. It is highly recommended to use this in the Software Download use case.



Caution
On some TC3xx devices, program flash blocks of different sizes are swapped. For example, a three Mega Byte block is swapped with a one Mega Byte block. In this case, the additional two Mega Byte of the first block cannot be used for program code in the Software Download use case. See [3] chapter 2 Memory Maps for the standard address map and the alternate address map for all Software Download able TC3xx devices.

3.5 Replace Logical Sector

The program flash on the Tc3xx devices is divided in program flash banks, which in turn are divided in 1 to 3 physical sectors. A physical sector is of size 1 Mbyte and consists of 64 logical sectors of size 16 Kbyte (see [3] Chapter 6.2.3.1). For each physical sector there exist two redundant logical backup sectors. In case of a hard fail in a logical sector, this can be replaced by one of these redundant backup sectors (see [3] Chapter 6.5.2.2 “Replace Logical Sector” command sequence). For the general usage of the feature a detailed application note is provided by Infineon upon request.

The service to replace a logical sector is implemented as a hardware specific request inside the vMem_30_Tc3xxInf01. Once the vMem_30_Tc3xxInf01 accepts the request to replace a sector it checks whether the requested sector can be replaced. The requested sector can only be replaced if there are still redundant sectors available and the sector is not already a replacement sector.



Caution
Once a logical sector is replaced this cannot be undone. The decision if a logical sector shall be replaced lies entirely with the user of this service.

The vMem only reports an error, i.e. job result is failed, but no detailed information about the type of error and which request caused the error. The API `vMem_30_Tc3xxInf01_GetLastErrorState` described in chapter 5.2.3, can be called to check the exact error. Use this API to determine if an error occurred that is relevant for the replace logical sector feature or due to wrong usage of the device (driver), which should not trigger the replace sector command.

3.6 Error Handling

3.6.1 Development Error Reporting

By default, development errors are reported to the DET using the service `Det_ReportError()` as specified in [2], if development error reporting is enabled (i.e. pre-compile parameter `VMEM_30_TC3XXINF01_DEV_ERROR_DETECT==STD_ON`).

If another module is used for development error reporting, the function prototype for reporting the error can be configured by the integrator but must have the same signature as the service `Det_ReportError()`.

The reported vMem_30_Tc3xxInf01 ID is 255.

Note that the standard services of a vMem driver and their corresponding service IDs are described in [2].

The following service IDs identify the extended hardware specific services which are described in 5.2. The following table presents the service IDs and the related services:

Service ID	Service
0xF0	`vMem_30_Tc3xxInf01_GetActiveAddressRegion()`
0xF2	`vMem_30_Tc3xxInf01_ReplaceLogicalSector()`
0xF3	`vMem_30_Tc3xxInf01_GetLastErrorState()`

Table 3-2 Service IDs



Note
All used error codes reported to DET are described in the corresponding table in [2], chapter Error Handling.

3.7 Hardware Software Interface

3.7.1 Data Memory Unit (DMU)

The vMem_30_Tc3xxInf01 accesses the registers of the DMU to check the flash status, flash operation and the error status. The registers are described in [3] in chapter 6.5.3.

The flash operations are performed with command sequences, except memory mapped reads. The Command Sequence Interpreter (CSI) of the DMU processes the command sequences. See [3] chapter 6.5.2. for a detailed description of the command sequences.

3.7.2 System Memory Unit (SCU)

For the feature GetActiveAddressArea the vMem_30_Tc3xxInf01 accesses the Alternate Address Control (SWAPCTRL) register of the SCU. Note that the vMem_30_Tc3xxInf01 only performs read access on this register. For a detailed description see [3] chapter 11.3.6.5.

3.7.3 User Configuration Block (UCB)

For the feature ReplaceLogicalSector the vMem_30_Tc3xxInf01 access the UCB REDSEC (Redundant Sectors) entries. Note that vMem_30_Tc3xxInf01 only performs read access on this UCB entries. For a detailed description see [3] chapter 6.8.3.3.

3.7.4 Compiler Abstraction

The vMem_30_Tc3xxInf01 knows two compilers and their abstraction: the TriCore GNU HighTec and the TriCore Tasking compiler. For all other compiler the integrator is responsible to define the required abstraction out of the vMem_30_Tc3xxInf01 component.

3.7.5 HSM Bridge

The vMem_30_Tc3xxInf01 accesses registers of the HSM Bridge module to get access to HOST peripheral, i.e. CSI of the program flash. The registers are described in [5], chapter 9.3. Usage of these registers is only needed when running from the HSM module of the TriCore.

Integration instructions on how to use the vMemTc3xxInf01 in context of the HSM is described in chapter 4.3.



Note
In context of HSM the register access through the OS peripheral API is not supported.

3.8 Safe Context: OS peripheral access API

Depending on the configuration of the parameter `vMemUsePeripheralAccessApi` the memory mapped registers are either accessed directly or via the APIs offered by the Microsar Operating System (OS). Refer to [4] for further information on the OS peripheral access API feature.

If the peripheral access API shall be used, the following OS Peripheral Regions must be created in the Microsar OS module in DaVinci Configurator.

Host Command Sequence Interpreter (HCSI)

Parameter	Value
Shortname	OS_MEM_AREA_VMEM_30_TC3XXINF01_REG_HCSI
Peripheral Region Start Address	0xAF000000
Peripheral Region End Address	0xAF0FFFFF

Table 3-3 Peripheral Region HCSI

Data Memory Unit (DMU)

Parameter	Value
Shortname	OS_MEM_AREA_VMEM_30_TC3XXINF01_REG_DMU

Parameter	Value
Peripheral Region Start Address	0xF8040000
Peripheral Region End Address	0xF807FFFF

Table 3-4 Peripheral Region DMU

System Control Unit (SCU)

Parameter	Value
Shortname	OS_MEM_AREA_VMEM_30_TC3XXINF01_REG_SCU
Peripheral Region Start Address	0xF0036000
Peripheral Region End Address	0xF00363FF

Table 3-5 Peripheral Region SCU

User Configuration Block (UCB)

Parameter	Value
Shortname	OS_MEM_AREA_VMEM_30_TC3XXINF01_REG_UCB
Peripheral Region Start Address	0xAF401800
Peripheral Region End Address	0xAF4019FF

Table 3-6 Peripheral Region UCB

4 Integration

This chapter gives necessary information for the integration of the MICROSAR vMem_30_Tc3xxInf01 into an application environment of an ECU.

4.1 Scope of Delivery

The delivery of the vMem_30_Tc3xxInf01 contains the files which are described in the corresponding chapters in [2].

In addition, the vMem_30_Tc3xxInf01 contains the following static file(s).

File Name	Description
vMem_30_Tc3xxInf01_Extended_Func.h	This file contains the types and declarations of the hardware specific functionality of vMem_30_Tc3xxInf01. It should be included by every user, that wants to call a hardware specific service.

Table 4-1 Additional static files

In addition, the vMem_30_Tc3xxInf01 contains another dynamic file which is generated by the configuration tool.

File Name	Description
vMem_30_Tc3xxInf01_Callout.h	This file contains the declarations of the callout functions of the vMem_30_Tc3xxInf01. It contains the declarations for locking/unlocking registers, that belong to other components.

Table 4-2 Additional Generated files

4.2 Critical Sections

To protect critical code against interruptions the vMem_30_Tc3xxInf01 uses the following exclusive area:

> `VMEM_30_TC3XXINF01_EXCLUSIVE_AREA_0`

The exclusive area covers only some statements and shall have a short runtime. The critical section is used vMem_30_Tc3xxInf01 internally and does not cover calls to other modules. vMem_30_Tc3xxInf01 does not nest critical sections.



Note
The vMem_30_Tc3xxInf01 uses the SchM (Schedule Manager) to enter and exit the critical section. The SchM is part of the component RTE. The vMem_30_Tc3xxInf01 includes the file “SchM_vMem_30_Tc3xxInf01.h” where the functions SchM_Enter_vMem_30_Tc3xxInf01_VMEM_30_TC3XXINF01_EXCLUSIVE_AREA_0() and SchM_Exit_vMem_30_Tc3xxInf01_VMEM_30_TC3XXINF01_EXCLUSIVE_AREA_0() must be provided.

4.3 HSM Integration

The vMem_30_Tc3xxInf01 supports the HSM core of the TriCore platform through a preprocessor macro, which must be used in context of the HSM.

Define

> VHSM_CODE

within your build-environment to allow the vMem_30_Tc3xxInf01 access to program flash from the HSM core.

5 API Description

For an interface overview please see Figure 2-1.

5.1 Type Definitions

The types needed for the extended hardware specific functionality defined by the vMem_30_Tc3xxInf01 are described in this chapter. For the definition of the standard vMem types see [2].

Type Name	C-Type	Description	Value Range
vMem_30_Tc3xxInf01_AddressRegionType	enum	The address regions defined by the AURIX TC3xx for grouping PFLASH banks. When Software Download is enabled, one of these groups of banks can be read and executed from, while the other group can have new code written to it.	VMEM_30_TC3XXINF01_ADDRESS_REGION_A
			VMEM_30_TC3XXINF01_ADDRESS_REGION_B
vMem_30_Tc3xxInf01_HWSpecificFuncID	enum	Index of hardware specific functionality in function pointer table.	VMEM_30_TC3XXINF01_ACTIVE_ADDRESS_REGION Function index of the hardware specific request vMem_30_Tc3xxInf01_GetActiveAddressRegion().
			VMEM_30_TC3XXINF01_REPLACE_LOGICAL_SECTOR Function index of the hardware specific request vMem_30_Tc3xxInf01_ReplaceLogicalSector().
			VMEM_30_TC3XXINF01_GET_LAST_ERROR_STATE Function index of the hardware specific request vMem_30_Tc3xxInf01_GetLastErrorState().
vMem_30_Tc3xxInf01_LockLevelType	enum	Specifies lock/unlock level.	VMEM_30_TC3XXINF01_SYSTEM Lock/unlock system/safety level.
			VMEM_30_TC3XXINF01_CPU Lock/unlock CPU specific level.
vMem_30_Tc3xxInf01_JobType	enum	Specifies the type of request which was executed last.	VMEM_30_TC3XXINF01_NO_JOB VMEM_30_TC3XXINF01_WRITE_JOB

Type Name	C-Type	Description	Value Range
			VMEM_30_TC3XXINF01_READ_JOB VMEM_30_TC3XXINF01_ERASE_JOB VMEM_30_TC3XXINF01_ISBLANK_JOB VMEM_30_TC3XXINF01_REPLACE_JOB
vMem_30_Tc3xxInf01_ErrorStateType	struct	This structure type must be used as data type when calling the hardware specific request vMem_30_Tc3xxInf01_GetLastErrorState.	vMem_30_Tc3xxInf01_JobType indicates the last request type.
			vMem_30_Tc3xxInf01_AddressType indicates the error state belonging to the last request.

Table 5-1 Type definitions of extended hardware specific functionality

vMem_30_Tc3xxInf01_ReplaceSectorType

This structure is the structure type needed for the hardware specific request to replace a logical sector vMem_30_Tc3xxInf01_ReplaceLogicalSector(). The user of this request must pass a pointer to a variable of this type.

Struct Elemenet Name	C-Type	Description	Value Range
TargetAddress	uint32	Start address of the sector that shall be replaced.	Must be the start address of a logical sector and be within range of configured address space.
RedundantSectorInfo	enum	Indicates if sector is replaceable or the reason why it is not replaceable.	VMEM_30_TC3XXINF01_NO_REDUNDANT_SECTOR_AVAILABLE The request to replace the sector cannot be executed because there is no redundant sector available.
			VMEM_30_TC3XXINF01_SECTOR_IS_REPLACEMENT_SECTOR The request to replace the sector cannot be executed because the requested sector is already a replacement sector.
			VMEM_30_TC3XXINF01_SECTOR_REPLACEABLE The request to replace the sector can be executed.

Table 5-2 vMem_30_Tc3xxInf01_ReplaceSectorType

5.2 Services provided by vMem_30_Tc3xxInf01

5.2.1 vMem_30_Tc3xxInf01_GetActiveAddressRegion

Prototype

`Std_ReturnType` **`vMem_30_Tc3xxInf01_GetActiveAddressRegion`**
`(vMem_30_Tc3xxInf01_InstanceIdType InstanceId,`
`vMem_30_Tc3xxInf01_DataPtrType ReqSpecificData,`
`uint32 SizeOfData)`

Parameter
InstanceId [in] — ID of the related vMem instance.
ReqSpecificData [in,out] — Pointer to a variable of type `vMem_30_Tc3xxInf01_AddressRegionType`. Contains the information about active address region on return.
SizeOfData [in] — Not needed for this request.
Return code
Std_ReturnType — E_OK request accepted, E_NOT_OK otherwise.
Functional Description
Returns the currently active address region. **Note:** This service must be called via the HwSpecificReq API of the vMemAccM. The corresponding function index is VMEM_30_TC3XXINF01_ACTIVE_ADDRESS_REGION. Via this service the user can obtain the information which address region is currently active. An address region is a group of PFLASH banks. An active PFLASH bank is currently used for code execution by the running application. In case of AURIX TC39x address region A consists of PFLASH0, PFLASH1 and PFLASH4. Address region B consists of the remaining banks PFLASH2, PFLASH3 and PFLASH5.
Particularities and Limitations
-
Call context
> TASK > This function is synchronous. > This function is reentrant.

Table 5-3 vMem_30_Tc3xxInf01_GetActiveAddressRegion

5.2.2 vMem_30_Tc3xxInf01_ReplaceLogicalSector

Prototype

`Std_ReturnType` **`vMem_30_Tc3xxInf01_ReplaceLogicalSector`**
`(vMem_30_Tc3xxInf01_InstanceIdType InstanceId,`
`vMem_30_Tc3xxInf01_DataPtrType ReqSpecificData,`
`uint32 SizeOfData)`

Parameter
InstanceId [in] — ID of the related vMem instance.
ReqSpecificData [in,out] — Pointer to a variable of type `vMem_30_Tc3xxInf01_ReplaceSectorType`. Struct member `TargetAddress` contains the address of the (defective) logical sector, that shall be replaced. Once the job is completed the struct

	member `RedundantSectorInfo` contains the information if the sector was replaceable, or if no redundant sector is available or if the sector is already a replacement sector.
SizeOfData [in]	Shall be the length of the `vMem_30_Tc3xxInf01_ReplaceSectorType`.
Return code	
Std_ReturnType	E_OK request accepted, E_NOT_OK otherwise.
Functional Description	
This service replaces a logical sector. **Note**: This service must be called via the HwSpecificReq API of the vMemAccM. The corresponding function index is VMEM_30_TC3XXINF01_REPLACE_LOGICAL_SECTOR. **Note**: The given address `TargetAddress` must be aligned to a (logical) sector. Via this service the user can replace a (failing) logical sector. It is up to the user to decide, if and when a sector should be replaced. If the request is accepted the vMem_30_Tc3xxInf01 first checks, whether the requested sector is replaceable (a sector can only be replaced, if it is not already a replacement sector, and if there is still a redundant sector available). If this is not the case the job result is JOB_FAILED. See chapter 3.5 for more information about the replace logical sector feature.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is asynchronous. > This function is not reentrant.	

Table 5-4 vMem_30_Tc3xxInf01_ReplaceLogicalSector

5.2.3 vMem_30_Tc3xxInf01_GetLastErrorState

Prototype	
`Std_ReturnType vMem_30_Tc3xxInf01_GetLastErrorState (vMem_30_Tc3xxInf01_InstanceIdType InstanceId, vMem_30_Tc3xxInf01_DataPtrType ReqSpecificData, uint32 SizeOfData)`	
Parameter	
InstanceId [in]	ID of the related vMem instance.
ReqSpecificData [out]	Pointer to a variable of type `vMem_30_Tc3xxInf01_ErrorStateType`. This structure will be filled with information about the last request and the corresponding error states.
SizeOfData [in]	Shall be the length of the `vMem_30_Tc3xxInf01_ErrorStateType`.
Return code	
Std_ReturnType	E_OK request accepted, E_NOT_OK otherwise.
Functional Description	
This service gets the content of the flash peripheral error register. **Note**: This service must be called via the HwSpecificReq API of the vMemAccM. The corresponding function index is VMEM_30_TC3XXINF01_GET_LAST_ERROR_STATE	

Via this service the user can check for detailed errors when the vMem reported the generic error state VMEM_JOB_FAILED to the user. This service reports always the latest error state of the last job. A possible use case is to check for the exact error condition when implementing the replace logical sector feature.
Particularities and Limitations
-
Call context
> TASK > This function is synchronous. > This function is not reentrant.

Table 5-5 vMem_30_Tc3xxInf01_GetLastErrorState

5.3 Services used by vMem_30_Tc3xxInf01

In addition to the services described in [2] the vMem_30_Tc3xxInf01 uses services provided by other components as in the following table. For details about prototype and functionality refer to the documentation of the providing component.

Component	API
OS (optional)	osReadPeripheral32
	osWritePeripheral32
SchM (Schedule Manager)	SchM_Enter_vMem_30_Tc3xxInf01_VMEM_30_TC3XXINF01_EXCLUSIVE_AREA_0()
	SchM_Exit_vMem_30_Tc3xxInf01_VMEM_30_TC3XXINF01_EXCLUSIVE_AREA_0()

Table 5-6 Services used by the vMem_30_Tc3xxInf01

5.4 Callout Functions

The vMem_30_Tc3xxInf01 defines callout functions. The declarations of the callout functions are provided by the BSW module, i.e. the vMem_30_Tc3xxInf01. It is the integrator's task to provide the corresponding function definitions. The definitions of the callouts can be adjusted to the system's needs. The vMem_30_Tc3xxInf01 callout function declarations are described in the following tables:

5.4.1 vMem_30_Tc3xxInf01_Unlock

Prototype	
`void vMem_30_Tc3xxInf01_Unlock (vMem_30_Tc3xxInf01_LockLevelType Level)`	
Parameter	
Level [in]	the requested lock level.
Return code	
void	none
Functional Description	
Unlock registers/ HW belonging to other components than the PFLASH driver vMem_30_Tc3xxInf01. vMem_30_Tc3xxInf01 is a PFLASH driver. For e.g. the endinit protection, access to registers/ HW belonging to another component may be necessary – this access has to be implemented outside of the	

vMem_30_Tc3xxInf01 component. If no unlocking is needed, the function shall be empty.
Particularities and Limitations
-
Call context
> TASK > This function is Synchronous > This function is Reentrant

Table 5-7 vMem_30_Tc3xxInf01_Unlock

5.4.2 vMem_30_Tc3xxInf01_Lock

Prototype	
`void` **`vMem_30_Tc3xxInf01_Lock`** `(vMem_30_Tc3xxInf01_LockLevelType Level)`	
Parameter	
Level [in]	the requested lock level.
Return code	
void	none
Functional Description	
Lock registers/ HW belonging to other components than the PFLASH driver vMem_30_Tc3xxInf01. vMem_30_Tc3xxInf01 is a PFLASH driver. For e.g. the endinit protection, access to registers/ HW belonging to another component may be necessary – this access has to be implemented outside of the vMem_30_Tc3xxInf01 component. If no locking is needed, the function shall be empty.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Reentrant	

Table 5-8 vMem_30_Tc3xxInf01_Lock

6 Configuration

Information on the configuration in general can be found in [2].

6.1 Configuration of Alternate Address Mapping

In this chapter the configuration for the Software Download use case is described in more detail.

As described in chapter 0 the TC3xx platform uses two address mappings to support Software Download. In the use case of Software Download, if users of vMem_30_Tc3xxInf01 always want to use the standard address map, i.e. the actual physical addresses, the alternate address mapping must be configured. This is done by referencing for every sector batch the corresponding sector batch in the alternate address mapping. This is the sector batch, which has the same start address in the alternate address mapping. The referenced sector batch must be identical regarding number of sectors, page size and sector size. Furthermore, the referenced sector batch must reference the original sector batch in return.

See [3] chapter 2 Memory Maps for the standard address map and the alternate address map for all Software Download able TC3xx devices.

The next figure shows an example configuration for TC39x devices. Let us consider the first sector batch ‘PF0_PF2_PS0_S0_S63’. As the name suggests, this is the first physical sector (PS0) of the flash module Program Flash 0 (PF0). Its physical start address is 0xA0000000, as described in the standard address mapping. In the alternate address mapping the flash module PF0 is swapped with the flash module Program Flash 2 (PF2). Therefore, the referenced sector batch (Alternate Map Sector Ref) is ‘PF2_PF0_PS0_S0_S63’, the first physical sector of the flash module PF2. The sector batch ‘PF2_PF0_PS0_S0_S63’ references the sector batch ‘PF0_PF2_PS0_S0_S63’ in return.

Note that only the first physical sector, i.e. the first Mega Byte, is configured for Program Flash 4, even though it consists of three Mega Byte. The reason for this is, that PF4 is swapped with PF5, which consists of only one Mega Byte. Therefore, the remaining two Mega Byte of PF4 cannot be used for program code in the OTA use case.

vMemSectors	Alternate Map Sector Ref	Number Of Sectors	Page Size [Byte]	Sector Size [kByte]	Start Address
PF0_PF2_PS0_S0_S63	PF2_PF0_PS0_S0_S63	64	32	16	0xA0000000
PF0_PF2_PS1_S64_S127	PF2_PF0_PS1_S64_S127	64	32	16	0xA0100000
PF0_PF2_PS2_S128_S191	PF2_PF0_PS2_S128_S191	64	32	16	0xA0200000
PF1_PF3_PS0_S0_S63	PF3_PF1_PS0_S0_S63	64	32	16	0xA0300000
PF1_PF3_PS1_S64_S127	PF3_PF1_PS1_S64_S127	64	32	16	0xA0400000
PF1_PF3_PS2_S128_S191	PF3_PF1_PS2_S128_S191	64	32	16	0xA0500000
PF2_PF0_PS0_S0_S63	PF0_PF2_PS0_S0_S63	64	32	16	0xA0600000
PF2_PF0_PS1_S64_S127	PF0_PF2_PS1_S64_S127	64	32	16	0xA0700000
PF2_PF0_PS2_S128_S191	PF0_PF2_PS2_S128_S191	64	32	16	0xA0800000
PF3_PF1_PS0_S0_S63	PF1_PF3_PS0_S0_S63	64	32	16	0xA0900000
PF3_PF1_PS1_S64_S127	PF1_PF3_PS1_S64_S127	64	32	16	0xA0A00000
PF3_PF1_PS2_S128_S191	PF1_PF3_PS2_S128_S191	64	32	16	0xA0B00000
PF4_PF5_PS0_S0_S63	PF5_PF4_PS0_S0_S63	64	32	16	0xA0C00000
PF5_PF4_PS0_S0_S63	PF4_PF5_PS0_S0_S63	64	32	16	0xA0F00000

Figure 6-1 Example configuration for TC39x

6.2 Configuration of Write and Erase Burst Feature

The vMem_30_Tc3xxInf01 supports two modes of writing and erasing. A standard mode where a single page can be written and a single sector can be erased during one flash access. To achieve a higher data throughput, when writing or erasing big amount of data the vMem_30_Tc3xxInf01 can use an internal burst mode. This burst mode enables the possibility to write multiple pages or erase multiple sectors during one flash access cycle.

6.2.1 Limitations for Erase Burst

The burst feature can be configured to erase one sector (16KB) and up to 32 sectors (512KB) simultaneously. For the erase burst feature to work correctly the vMemSectors must only contain logical sectors from within the same physical sector (for the description of what a physical sector and/or a logical sector in context of the Tc3xx devices is, see [3]). This is due to hardware restrictions.

The EraseBurstSize should be configured after evaluating the use case when using the vMem_30_Tc3xxInf01, since the vMem_30_Tc3xxInf01 only executes erase requests, which have sector length or EraseBurstSize.

For instance, if a length of 128KB is erased very often, but a EraseBurstSize of 512KB is configured, the burst feature will not be used. In that case it is recommended to configure the EraseBurstSize to 128KB, to make best use of the burst feature.

6.2.2 Limitations for Write Burst

The Write burst works similar as the erase burst feature. The WriteBurstSize can be configured with 256Byte. Other sizes are not supported by hardware.



Note
PageBurstRamAlignment should always be configured as 1.

7 Glossary and Abbreviations

7.1 Glossary

Term	Description
AURIX TC3xx	Infineon Platform family, a range of 32-bit multicore microcontrollers based on the Infineon Tricore Architecture.
vMemAccM	Vector Memory Access Manager: the most likely user of the vMem driver. The vMemAccM partitions and distributes memory access jobs from multiple users to the various vMem drivers.

Table 7-1 Glossary

7.2 Abbreviations

Abbreviation	Description
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
BSW	Basis Software
DET	Development Error Tracer
DMU	Data Memory Unit
ECU	Electronic Control Unit
HIS	Hersteller Initiative Software
HSM	Hardware Security Module
MICROSAR	Microcontroller Open System Architecture (the Vector AUTOSAR solution)
SCU	System Memory Unit
OS	Operating System
OTA	Update Over The Air
RTE	Runtime Environment
SWS	Software Specification

Table 7-2 Abbreviations

8 Contact

Visit our website for more information on

- News
- Products
- Demo software
- Support
- Training data
- Addresses

www.vector.com